Zurich ServiceNow AI Platform Capabilities

Last updated: January 16, 2026

Some examples and graphics depicted herein are provided for illustration only. No real association or connection to ServiceNow products or services is intended or should be inferred.

This PDF was created from content on docs.servicenow.com. The web site is updated frequently. For the most current ServiceNow product documentation, go to docs.servicenow.com.

Company Headquarters
2225 Lawson Lane
Santa Clara, CA 95054
United States
(408)501-8550

IntegrationHub ETL

Use the IntegrationHub ETL store app to create and manage ETL transform maps, which integrate third-party data into the CMDB or into non-CMDB tables without compromising the integrity of data. IntegrationHub ETL provides a simplified user interface that guides you through the integration process end-to-end, including a test integration run of sample data.

The IntegrationHub ETL (sn_int_studio) plugin provides the IntegrationHub ETL functionality.

- Use the CMDB Integrations Dashboard to track progress, results, and errors associated with using custom integrations created in IntegrationHub ETL. The CMDB Integrations Dashboard is included in the Integration Commons for CMDB store app.
- Watch the IntegrationHub ETL | Importing resources into the CMDB video for an introduction and walk through of the IntegrationHub ETL tool.

Request apps on the Store

Visit the ServiceNow Store website to view all the available apps and for information about submitting requests to the store. For cumulative release notes information for all released apps, see the ServiceNow Store version history release notes.

Roles required

Users with the cmdb_inst_admin role can use IntegrationHub ETL to create integrations, or customize a pre-existing integration provided by ServiceNow or a vendor at the ServiceNow Store. A vendor can create a new integration and provide it as an application for anyone to use.

Support for non-CMDB tables

Starting with the Zurich release, IntegrationHub ETL supports the integration of third-party data into some non-CMDB tables. IntegrationHub ETL supports those non-CMDB tables that are supported by Identification and Reconciliation (IRE). For details about which non-

CMDB tables are supported and any needed configuration, see IRE support for non-CMDB tables.

Supported non-CMDB tables are available in IntegrationHub ETL when specifying classes, conditional classes, class associations, and reference sources in mapping definitions. However, there are some differences between using CMDB classes and non-CMDB tables in IntegrationHub ETL:

- Specifying class associations isn't mandatory for non-CMDB tables.
- Adding relationships doesn't apply to non-CMDB tables.
- Class associations for a non-CMDB table is based on a reference field instead of a CMDB relationship.

> **Note:** Although the IntegrationHub ETL user interface and accompanying documentation references CMDB and CMDB elements, most of those references also apply to supported non-CMDB tables.

Process

The two key components that IntegrationHub ETL uses for processing are:

- Robust Transform Engine (RTE): Used to transform raw source data that is stored in staging tables, into the data that is mapped and integrated into the CMDB. RTE uses ETL transform maps that were created for the integration during data transformation.
- Identification and Reconciliation engine (IRE): Used as a centralized framework for identification and reconciliation processes across different data sources. IRE processes help maintain data integrity in the CMDB and in supported non-CMDB tables.

IntegrationHub ETL uses RTE and IRE which work together to process and integrate data. Data is first imported from a data source, and is then stored in temporary staging tables in Import Sets systems. Using the data in the staging tables and the ETL transform map created by IntegrationHub ETL, RTE creates IRE payloads which are then processed by IRE. IRE applies reconciliation processes to avoid potential problems such as duplicate CIs, ensuring that the CMDB or non-CMDB tables remain healthy, and then integrates the resulting data.

When you create an integration, you import source data, transform data if needed, and select target CMDB classes (or non-CMDB tables) and attributes to map the data to. Eventually, you run an integration test of the sample data, using your settings in the IntegrationHub ETL. You can then preview the integration test results and adjust any settings before scheduling recurring integration runs for large data sets. If you develop and test the ETL transform map on a development instance, then you can test and adjust the configuration before implementation on a production instance.

For example, you can integrate data from SCCM (Microsoft System Center Configuration Manager).

Refer to the community page IntegrationHub-Extract Tranform Load (IH-ETL) is GA in ServiceNow store for an overview of IntegrationHub ETL, including its components and workflow.

Guided Setup

A guided setup organizes all the tasks in the correct order, tracks the completion of tasks, and enforces any task dependencies. Tasks that depend on the completion of other tasks, are enabled or disabled as you step through the tool and complete tasks.

Read-only mode

When opening a Service Graph Connector in which IntegrationHub ETL isn't detecting any incoming data from the data source, the integration is available in read-only mode. In read-only mode you can access all the guided setup tasks on the ETL Transform Map Assistant page. You can examine all the settings and definitions in the integration even though it isn't populated with actual data. However, you can't make any updates to a read-only connection.

Read-only mode is useful for studying an existing connection for the purpose of creating a new connection that is similar to the read-only connection. The read-only mode can also assist in troubleshooting issues with the connection.

IntegrationHub ETL and Import Sets

Using IntegrationHub ETL and ETL transform maps has the following advantages over using Import Sets and transform maps:

- Identification and Reconciliation Engine (IRE) processes are incorporated into the IntegrationHub ETL so all data is automatically processed by IRE as part of the integration. Using Import Sets and transform maps does not provide a simple way to apply IRE processes.
- IntegrationHub ETL uses guided setup which provides guidance and a simple user interface for the entire process of integrating third-party data.
- IntegrationHub ETL includes an integration test for a small data set using the new ETL transform map. This test lets you review the results and adjust configuration settings before scheduling recurring integrations.

Terms

The following terms are associated with the IntegrationHub ETL:

CMDB application

Name of the third-party vendor such as SCCM 2019. A CMDB application has two associated attributes: Name and Discovery Source. When creating a new integration, ensure to configure a discovery source for the CMDB application that you plan to use, before using the IntegrationHub ETL.

Data source

The source feed, such as SCCM 7.0 Computer Identity, where the raw source data is imported from. If you use various REST endpoints for different types of data, then each REST endpoint is associated with its own data source and an ETL transform map.

ETL transform map

The output generated by IntegrationHub ETL. You can integrate third-party data into the CMDB or into non-CMDB tables using an ETL transform map which is configured for the respective integration.

Source data

Original, raw data that have been imported into IntegrationHub ETL. Source data can be used in its original form, or you can transform the data before mapping and integration.

Transform

An operation, that you can apply to a specific data column to transform the data values. For example, to transform the format of the data values. Use transforms to standardize data formats and meet other system requirements.

Transformed data

Some of the source data might not be compliant with the requirements of its target CMDB attributes and classes or non-CMDB tables. In those cases, you can apply various types of transforms to the source data, before mapping the data to the target CMDB classes and attributes or non-CMDB tables. Transforms, can for example convert data format, replace values, and concatenate values from multiple data columns.

Each CMDB application can have multiple connections for retrieving raw data. Each connection that is used to retrieve a certain type of data, has its own pair of data source and an ETL transform map. Therefore, one CMDB application can have multiple ETL transform maps, and each of those ETL transform maps is associated with a single Data Source.

For example:

CMDB Application	ETL Transform Map	Data Source
SCCM	SCCM Computer Identify	/sccm/2019/comp
	SCCM Disk	/sccm/2019/disk
	SCCM Application	/sccm/2019/appl

Nested data payloads

To process nested data payloads, you must first ensure that the data source that is used for the integration, is set with the Data in single column option. With that setting, you can correctly represent nested data in a JSON payload which IntegrationHub ETL then processes as nested data, rather than as flat data.

Sample of nested data:

```
{
            "u_computer_fqdn": "computer2-fqdn",
            "u_computer_id": 2,
            "u_computer_ip": "computer2-ip",
            "u_computer_location": "PDX",
            "u_computer_mac": "computer2-mac",
            "u_computer_name": "nested-payload-computer2"
,
            "u_computer_os": "computer2-os",
            "interfaces": [
                {
                    "u_interface_ip": "computer2-eth1-ip"
,
                    "u_interface_mac": "computer2-eth1-ma
c",
                    "u_interface_name": "computer2-eth1",
                    "ip": ""
                },
                {
                    "u_interface_ip": "computer2-eth2-ip"
,
                    "u_interface_mac": "computer2-eth2-ma
c",
                    "u_interface_name": "computer2-eth2",
                    "ip": {
                        "u_ip_address": "computer2-eth2-i
p",
                        "u_mac_address": "computer2-eth2-
mac"
                    }
                }
            ],
            "software": [
                {
                    "u_software_name": "computer2-softwar
e2",
                    "u_software_version": "computer2-soft
ware2-1.0",
                    "instance": {
                        "u_software_instance_name": "comp
uter2-software1-instance"
                    }
                },
```

```
                {
                    "u_software_name": "computer2-softwar
e2",
                    "u_software_version": "computer2-soft
ware2-2.0",
                    "instance": {
                        "u_software_instance_name": "comp
uter2-software2-instance"
                    }
                }
            ]
        },
```

You can view the layers of nested data in a separate panel in IntegrationHub ETL, apply transforms, map, and integrate that data into the CMDB.
When creating a nested data JSON payload, the following restrictions apply:

- Field names must start with a letter (between A-Z or a-z) or with '_', and must only contain letters (between A-Z or a-z), digits (0-9), or the '_' character.

  For example, a field name can't contain special characters such as *, [,], #, $, spaces, and dot.

- Field names can't be "temp" or "object", which are reserved for internal use.
- Consistently throughout the payload, you must use an array or an object to represent data in a specific level, regardless of the number of items in the level. If you use an array for multiple items in one object, you must also use an array to represent a single item in other objects.

For a demo about working with nested payload data, watch the Integration Hub - ETL nested payload feature demo video on the ServiceNow YouTube channel.

Related reference

- Teams related list

Create an ETL transform map

IntegrationHub ETL provides a guided setup which walks you through the completion of all necessary tasks for creating an ETL transform map for a specific integration.

Guided setup

Guided setup organizes all the tasks in the correct order, tracks the completion of tasks, and enforces any task dependencies. Tasks that depend on the completion of other tasks, are enabled or disabled as you step through the tool and complete tasks.

Use guided setup on the ETL Transform Map Assistant page to complete the following tasks.

Import source data and specify basic details

Provide basic details for the integration, such as the source of the data that you want to integrate into CMDB, and import the source data.

Before you begin

The data source that you plan to select for the ETL Transform Map must exist in the same application scope as the one being used in the current session.

When you open an ETL transform map, by default the map is not validated. You can enable this validation step by adding the system property sn_int_studio.validation.enabled to the System Properties [sys_properties] table and then setting it to **true**. After validation is complete, you choose how to handle validation errors.

Role required: cmdb_inst_admin

Procedure

1. Navigate to **All > Configuration > IntegrationHub ETL**.
   The landing page of the IntegrationHub ETL lists all integrations that exist in the system, including integrations that were downloaded from the ServiceNow Store. Starting with IntegrationHub ETL v3.2, integrations are grouped by the CMDB Application value, in which case expand the respective group to locate an integration.

2. Click the **Name** of an integration to view or modify, or click **Create new**.
   If the system property sn_int_studio.validation.enabled is set to **true**, then IntegrationHub ETL validates the ETL transform map that you are loading. If there are any validation errors, the Invalid Mapping Data Detected dialog box appears, listing all the specific errors that were detected. You can choose to delete the invalid mappings and continue with only the valid mappings, or you can choose to keep the invalid mappings. However, notifications about invalid mappings will continue to appear as you continue to work with the integration. The system detects errors such as:

   - Missing source or target fields in corresponding Robust Transform Engine (RTE) field mappings records
   - Missing table columns in an import set

     Note: In this situation, any corresponding metadata records in RTE are no longer valid and are automatically deleted. Records such as field mappings and transform operations that are associated with the missing table columns in the import set, are deleted.

   - Missing an Identification and Reconciliation Engine (IRE) lookup rule for a lookup class

3. On the ETL Transform Map Assistant page, in the Specify Basic Details section of the guided setup, select the **Import Source Data and Provide Basic Details** task.

4. Fill out the form.

Field	Description
CMDB Application	The CMDB application associated with the ETL transform map. You can select **Add new**, which adds the **CMDB Application** and

Field	Description
	the **Discovery Source** fields for the new CMDB application.
Name	Name of the ETL transform map.
Description	Description of the integration.
Data Source	List of all data sources in the system. **Note:** Be cautious in subsequently modifying the data source as it can result in substantial changes to the data integration. Aligning to the import set table of the new data source might require the removal of columns and associated transforms, or the addition of new columns. IntegrationHub ETL validation processes will detect any required updates and let you agree or reject these updates.
Sample Import Set	An Import Set that is associated with the specified **Data Source**. A subset of that Import Set data is used to preview source data. Select the **Auto-pull a new import set** option to pull a new Import Set of the associated data source. Starting with IntegrationHub ETL v3.2, if no Import Set is specified,

Field	Description
	then the map is loaded and is automatically set to be in read-only mode. You can review configurations in the map but can't edit mappings or transforms.
Preview Size Override	Number of data records that are loaded and used as sample for the preview for this transform map. If set, this custom setting overrides the value of the sn_int_studio.preview.size system property, and applies only to the current transform map. If **Load Complete Schema** is disabled, then the nested data structure for the map is generated based only on the specified number of records that are loaded. Field available starting with IntegrationHub ETL v3.2.
Load Complete Schema	Enable or disable loading the entire data schema for generating the data structure for the map. When disabled, the nested data structure for the map is generated based only on the number or records loaded as sample records for preview. The number of records loaded is determined by either the

Field	Description
	Preview Size Override setting, or by the global system property sn_int_studio.preview.size. Field available starting with IntegrationHub ETL v3.2.
CMDB Application	Name of a new **CMDB Application**. Appears if you set **CMDB Application** to **Add new**.
Discovery Source	Discovery source associated with a new **CMDB Application**. Appears if you set **CMDB Application** to **Add new**.

5. Click **Save** to save the current changes or **Mark as Complete**.

   A time stamp appears in the header when you click **Save**, which remains for the duration of the IntegrationHub ETL session for the ETL transform map. When you re-enter the session or switch between ETL maps, the time stamp disappears.

Related topics

- Create an Import Set data source

Preview and prepare data

Review sample records of raw source data, which will be integrated into the CMDB. Transform and prepare data to align with the target classes and attributes, if needed.

Before you begin

The number of records in the sample data is globally determined by the system property sn_int_studio.preview.size, which is set to 100 by default. The maximum number of records in the sample data that IntegrationHub

ETL can process is 10,000. If you set that property above the 10,000 limit, then IntegrationHub ETL will only process up to 10,000 records and a message will appear to that effect.

Starting with IntegrationHub ETL v3.2, you can override the value of the sn_int_studio.preview.size property by setting the **Preview Size Override** field on the Import Source Data and Provide Basic Details form, per map.

To process nested data from a nested payload, the respective data source must be set with the **Data in single column** option.

Role required: cmdb_inst_admin

About this task

Review the values in the data columns of the sample data and identify columns that do not align with the requirements of the intended target classes and attributes. You can transform data, for example, by converting the data format, replacing values, and concatenating data columns. You can apply transformations one on top of another, creating a chain of data transformations. You can also set a data column to be ignored in the mapping and integration process.

> **Note:** To set a CMDB attribute to be empty, use the string '<EMPTY_STRING>'.

Columns for nested data appear alongside the rest of the data, with a **Nested Objects** notation in the data column header. The count of nested data items per object appear with a link which lets you drill to deeper levels of the nested data. To show the data structure of nested data in a separate panel, enable the **Show data structure** option.

The Data Structure panel has two options for displaying nested data:

- **Tree**: Nested data grouped by objects, where each object node corresponds to a record entry in the source data. Expand object nodes to show all nested data for the record.
- **Collection**: Nested data grouped by the top-level object (by default) and then by nested data items such as software. Expand a node such as software, to show which software is installed on each computer.

You can navigate through the levels of nested data in the Data Structure panel, the breadcrumbs path, or through number links that appear in

the source data itself. Your selections and the data that appears are kept synchronized between all views of the nested data, regardless of navigation.

For a demo about working with nested payload data, watch the Integration Hub - ETL nested payload feature demo video on the ServiceNow YouTube channel.

Procedure

1. Navigate to **All > Configuration > IntegrationHub ETL**, and click the **Name** of an integration.
   The landing page of the IntegrationHub ETL lists all integrations that exist in the system, including integrations that were downloaded from the ServiceNow Store.

2. In the ETL Transform Map Assistant page, in the Prepare Source Data for Mapping section of the guided setup, select **Preview and Prepare Data**.

3. (Optional) Select **Show data structure** to open the Data Structure panel which shows the structure of nested data. In the Data Structure panel, you can drill down through the levels of nested data.

4. (Optional) Select the action menu for a column and then select a Sort operation.

5. Select the action menu for a column and then select **Group by** to group the data by the respective column. Select **Ungroup** to undo the grouping operation.

6. (Optional) Click **New Transform** and then select **Use Source Column** . Or, select the action menu for a column, and then select **New Transform** to transform the selected column.

   You can't create new transforms for nested objects at this top-level view of the data. A nested object column contains number links which indicate the number of nested items for the record. To create a new transform for nested objects, click that number link to drill down to the actual nested data. Alternatively, navigate in the Data Structure panel to the nested object for which you want to create a transform.

A transform of nested data can reference parent objects of the nested data being transformed. Using the sample payload for nested data as an example, a transform for an interface object can reference the parent computer object but can't reference a software object.

a. In the New Transform sidebar on the right, select a **Transform Type** and modify the **Transform Description** if appropriate.
For more details about transform types, see Transform types in IntegrationHub ETL.

b. (Optional) Select **Hide initial column used for this transform** to hide from the current view all the columns that were used for this transform.
This setting is temporary for the current session, and if you refresh the page, the hidden column reappears. To show a hidden column, you can also click the gear icon on the banner frame. Then, move the hidden column from the Available to the Selected list and click **OK**.

c. Select or verify the **Input Column** whose values are being transformed.

d. (Optional) Modify the **Output Column Name** for any of the columns that will be added with the transformed values.

e. Click **Apply**.
A new column with the transformed values appears, placed in alphabetical order based on the output column name. If you used the suggested output column name, then the new column appears to the right of the input column.

f. Review the transformed data and adjust any transforms, if needed.

7. (Optional) To apply the 'Set Fixed Value Column' transform:

a. Click **New Transform** and then select **Set Fixed Value Column**.

b. In the Set Fixed Value Column sidebar, enter a **Column Name** and a **Column Description** for the new column. Then, set **Assign Column Value** to the value that is fixed for the new column.

c. Click **Apply**.

8. (Optional) Select the action menu for a column, and then select **Ignore in Mapping** to exclude the column from mapping and integration in the current session.

   In a subsequent session, the **Ignore in Mapping** setting does not apply, and the column will be included in mapping.

   You can click **Include in Mapping** to undo the **Ignore in Mapping** setting for the column.

9. (Optional) Select the action menu for a column, and then select **Delete This and Downstream Columns**. This delete action deletes the column along with any columns that were added using this column as an input column.

10. (Optional) Click **New Transform** and then select **Table Lookup** which lets you specify a table to look up and extract additional values from. Fill out the fields in the Table Lookup sidebar on the right.
    Values from the specified lookup table are matched with the mapped data. For the records that match, the specified values from the lookup table, are added as a column, to the data that is being prepared for mapping.

Table Lookup

Field	Description
Lookup Table	Table to use for matching with the data that is being mapped. When records from the lookup table and the mapped data satisfy the **Look Up Condition**, then specified values from the **Lookup Table** are extracted from the respective record and added to the mapped data.
Lookup Condition	A set of pairs of column conditions. Each pair specifies a column in the lookup table and a column in the mapped

Field	Description
	data, which are attempted to be matched. • **If values of target table column**: The column in the target table to match to a column in the mapped data. • **Match values of source data table**: The column in the mapped data to match to a column in the lookup table. You can add multiple pairs of columns to match on.
Lookup Condition	Values to extract from the **Lookup Table** when there is a match with the mapped data. **Then output values from the following columns**: The lookup table columns to extract values from, when values from the lookup table and the mapped data, satisfy the **Lookup Condition**. You can specify multiple lookup table columns to extract values from. For every column that you specify, a corresponding **Output Column Name** field automatically appears. Specify a label for the column that will be added with the extracted values.

Field	Description
Output Column Name	A label for the column that will be added to the mapped data, with the values extracted from the lookup table. An **Output Column Name** field is automatically added for every column that you specify in **Then output values from the following columns**.

11. Review the data and ensure that the intended set of data to be integrated is transformed, correctly formatted and prepared for import.
12. Click **Mark as Complete**.

Result

Data is prepared when the set of source data columns and transformed columns that you want to integrate, meet any formatting and other value requirements of the target CMDB classes and attributes. These columns are then ready to be mapped and integrated to CMDB classes and attributes.

About mapping data columns to CMDB classes and attributes

There are several requirements and guidelines for mapping source data to target CMDB classes and attributes. Also, there is an option of deactivating class mappings while preserving the settings for an easy reactivation. Review these concepts to ensure proper processing by the Identification and Reconciliation Engine (IRE).

Required mappings

You must map data to all required attributes of the target class in addition to mapping to attributes that are not configured as required. Also, the following two fields appear by default and you cannot delete them:

Source Native Key

IRE uses to uniquely identify a record and for building relationships and references. Also, improves performance of insert and update operations. When processing a payload, IRE generates an error if this field is empty.

Source Recency Timestamp

IRE uses to identify records that are older than the current record and therefore can be ignored, to help resolve conflicting attribute values. If a value is provided, it is used only if it is later than the value that is currently stored in the CMDB. If a value is not provided, IRE updates the attribute with the current timestamp.

The following system properties let you modify how IRE uses the source_recency_timestamp value in a payload to update the last_scan attribute in the Source [sys_object_source] table:

- glide.identification_engine.skip_updating_last_scan_if_older
- glide.identification_engine.ire_message_listener_skip_updating_last_scan_to_now

For more information about how IRE uses source_native_key and source_recency_timestamp for CI identification, see Identification and Reconciliation engine (IRE).

Conditional class

A conditional class lets you map different sets of data records to different target classes according to specific column values, or the status of a specific plugin.

For example, if a display name contains 'Windows', then 'Windows Server' is selected as the target class. But if the display name contains 'Linux', then 'Linux Server' is selected as the target class. For records that do not meet any of these conditions (display name does not contain 'Windows' nor 'Linux'), 'Server' is selected as the target class.

Associated class

An associated class lets you select the CMDB class to be associated with a target non-CMDB table. Setting an associated class is required for IRE processing if the non-CMDB table is not configured for IRE processing. For

a non-CMDB table that is supported and configured for IRE processing, setting an associated class is optional. See IRE support for non-CMDB tables for more information.

The software Instance is a non-CMDB class but it does not have IRE rules associated with it. So, things we said about it here pre-Utah are still valid. But for non-CMDB classes with IRE rules it's not mandatory to have an association. For example "If the target class for mapping is a non-CMDB class with a reference to a CMDB class, you must select the CMDB class to associate the non-CMDB target class with" non-CMDB class with IRE rules Instead of "you must" it should be. "You can". Same with the Example it's not valid for non-CMDB with IRE rules.

If the target class for mapping is a non-CMDB class with a reference to a CMDB class, you must select the CMDB class to associate the non-CMDB target class with. A non-CMDB class refers to a class, such Serial Number [cmdb_serial_number], that does not extend the Configuration Item [cmdb_ci] class. The Related Entry [cmdb_related_entry] class might contain multiple CMDB class associations for the same non-CMDB class. Therefore, select the appropriate association to allow IRE processes to update the target non-CMDB class.

For example, the Related Entry [cmdb_related_entry] class has a record which associates the non-CMDB Software Instance [cmdb_software_instance] class with the CMDB Software Package [cmdb_ci_spkg] class. If you select Software Instance as a target class, you must associate the Software Instance class with the Software Package [cmdb_ci_spkg] class.

Deactivating class mappings

When you edit an ETL transform map, provided by a Service Graph Connector for example, you can delete a class mapping to prevent the class from being populated when the integration runs. However, if you later decide to populate that class, you must readd that class and reconfigure all the class mappings. Instead, you can deactivate a class mapping to temporarily ignore the class during the integration run, while preserving all of its mapping configuration. A class that you choose to deactivate is grayed out in the user interface but you can continue and edit the class mappings. Later, you can reactivate a class mapping to enable populating the class, without needing to reconfigure the class mappings.

Some classes that you choose to deactivate, trigger an automatic deactivation of additional classes that you did not directly choose to deactivate. Which classes are automatically deactivated, depends on the class that you chose to deactivate. For example, whether the class has dependent relationships or associated classes. Those automatically deactivated classes:

- Appear in light gray in the user interface and you can't reactivate them.
- Are automatically reactivated when you reactivate:
  - The class that you initially deactivated which triggered the automatic deactivation
  - Any class that the deactivated class depends on

All classes that you directly deactivate mappings for and the resulting class mappings that are automatically deactivated, are not populated when the integration runs. Also, any relationships and lookup tables associated with those classes, are not populated when the integration runs.

Class mapping and other deactivation scenarios:

- Deactivate a class which no class depends on and which has no associated classes:

  Triggers an automatic deactivation of any lookup rules and relationships associated with the deactivated class.

- Deactivate a lookup rule, such as serial number, within a class mapping:

  Does not trigger any automatic deactivations.

- Deactivate a CMDB class which is associated with a non-CMDB class:
  - Triggers an automatic deactivation of the associated non-CMDB class.
  - Deactivating the non-CMDB class, does not impact the associated CMDB class.
-

Deactivate a class with dependent relationships (Applies only if the dependent relationship exists in IntegrationHub ETL):

- Triggers an automatic deactivation of any class that has a single dependent relationship with the deactivated class.
- If a class has multiple dependent relationships, then it is automatically deactivated only when you deactivate all of the dependent on classes.

  For example, a scenario in which the File System class has dependent relationships with both, the Computer and a Server class. If you deactivate the Computer class, the File System class is not automatically deactivated. Only if you also deactivate the Server class, the File System class is automatically deactivated.

- Deactivate a conditional class or a class mapping within a conditional class:
  - Deactivating or activating a conditional class, triggers an automatic deactivation or activation of all conditional class mappings within the conditional class.
  - Deactivating a class mapping within a conditional class: Prevents the deactivated class from getting populated during integration runs. However, the associated 'If', 'Else if', or 'Else' conditions themselves remain in effect within the condition of the conditional class. For example, if you deactivate the following class mapping:

    [If] [operating_system] [contains] [Linux] Then [Class] [is] [Linux Server].

    Then, the Linux Server class is not populated, but the **[If] [operating_system] [contains] [Linux]** condition is in effect.

Map data columns to CMDB classes and attributes

Choose target classes and attributes in the CMDB to map source data columns to. You can map a data column to a specific target class, or add conditions so that the choice of target class depends on specific data values.

Before you begin

Role required: cmdb_inst_admin

About this task

Data columns that you map can be either source data columns which were not transformed, or transformed data columns. For example, to integrate a data column into the Computer and Software Package classes, select those classes as target classes and then map data columns into specific attributes in those classes.

When you configure mapping for a class, relationship, or a lookup rule, those items are always initially set as activated. For details about the results of deactivating mappings, see Deactivating class mappings.

> **Note:** Changing a class impacts any mappings that were already configured for the class, sometimes deleting those mappings. Details about the affected mappings and the impact, appear in the Affected mappings dialog box before you proceed with the class change. However, these details appear only when the change is from a CMDB class to another CMDB class or from a non-CMDB class to another non-CMDB class.

Procedure

1. Navigate to **All > Configuration > IntegrationHub ETL**, and click the **Name** of an integration.
   The landing page of the IntegrationHub ETL lists all integrations that exist in the system, including integrations that were downloaded from the ServiceNow Store.

2. In the ETL Transform Map Assistant page, in the Map Data to CMDB and Add Relationships section of the guided setup, select **Select CMDB Classes to Map Source Data**.
   Attributes that are configured as required in the platform, are noted, and you must map a data column to each of those attributes.

3. Click **Add Class** to add a target class to map to, or click **Edit Class** to edit a class.

   a. In the Add Class dialog box, select a CMDB **Class**.

b. Click **Save**.

c. (Optional) Set the **Activate/Deactivate Mapping** toggle switch for a class, to on or off. If the Affected class mappings dialog box appears, review the list of affected classes, and then click **Proceed**.
When you add a non-CMDB class, it is initially deactivated and the **Activate/Deactivate Mapping** toggle switch is disabled, until you add an associated class that is active.

4. Click **Add Conditional Class** and then in the **Add Conditional Class** dialog box, specify the conditions that must be met for data to be mapped to different target classes.

   a. **Collection** is automatically set to the data branch in the hierarchy which is associated with the lowest-level attribute. You can modify the value to the data branch from which you want to map data from, which must be at a higher level in the same data branch of the hierarchy.

   b. In the **If** drop-down list, select attribute conditions that data values must meet, or enter `plugins` in the search box and specify a plugin condition. You can then specify that the rest of the records, which did not match any conditions, are mapped to yet a different target class. Data records will be mapped to different target classes according to the conditions met.

      When processing nested data, a prefix denotes the first level in the nested hierarchy for attribute items.

      Note: When you select a non-CMDB class, it is initially deactivated and the **Activate/Deactivate Mapping** toggle switch is disabled, until you add an associated class that is active.

   c. Click **Save**.

   d. (Optional) Set the **Activate/Deactivate Mapping** toggle switch for a conditional class, to on or off. If the Affected class mappings dialog box appears, review the list of affected classes, and then click **Proceed**.

   e. (Optional) Click **Edit Class** to edit the settings of a conditional class. In the Edit Conditional Class dialog box, set the **Activate/**

Deactivate Mapping toggle switch for a class mapping, to on or off. Click **Save**, and if the Affected class mappings dialog box appears, review the list of affected classes and then click **Proceed**.

- A deactivated class is not populated during integration runs, however, this doesn't affect the associated condition. The 'If', 'Else if', and 'Else' conditions themselves remain in effect within the condition of the conditional class and matching CIs are filtered accordingly.
- The toggle switch of the conditional class reflects the summary of the states of all the conditional class mappings within the conditional class. If at least one of the conditional class mappings is activated, then the toggle switch of the conditional class appears as activated. Otherwise, the toggle switch of the conditional class appears as deactivated.

5. For a non-CMDB class, click **Add Associated Class** to associate the non-CMDB class with a CMDB class and to enable the **Activate/ Deactivate Mapping** toggle switch.. Or, click **Edit Associated Class** to edit an already associated class.

   a. In the Add Associated Class dialog box, select a CMDB class. The list includes all entries in the Related Entry [cmdb_related_entry] class for the specified non-CMDB table (deactivated classes are not included).

   b. Click **Add**.

   c. (Optional) Set the **Activate/Deactivate Mapping** toggle switch for an associated class, to on or off.

      Note: If an associated class was not added or is deactivated, then the **Activate/Deactivate Mapping** toggle switch is disabled.

6. Click **Set Up Mapping** to configure mapping for a newly added class, or click **Edit Mapping** to edit a mapping.

   a. To map, drag data columns from the Data sidebar on the right, to CMDB target attribute on the left side of the mapping page.

Or, click the  icon to search and select data columns for the mapping.
When mapping nested data:

- Data columns in the Data sidebar appear in a tree format that represents the structure of the nested data. Each attribute is associated with sample data for the attribute.
- Transformed columns are noted by a cyan-shaded dot.
- All mappings to a specific CMDB class must be from the same source branch in the nested data. Only the branch from which you selected the first column to map, is valid for selecting columns in subsequent mappings.

  This restriction applies differently when mapping to attributes in lookup tables. All mappings to attributes in a lookup table also must be from the same source branch. However, that source branch can be different than the source branch you used with non-lookup tables.

  Note: You can work around this restriction by using the Copy transform in the data preparation step, to copy attributes from a parent level to a child level. Prepare the data so that all the attributes that you want to map, are at the same level.

- When you drag a column to map from the Data sidebar, the fields of CMDB target attributes that are valid for the mapping, are highlighted by a green frame. If you attempt to drop a column in an invalid target attribute, the respective field is highlighted by a red frame and an error appears.

b. Click **Add Attribute**. Then, in the Add Attribute dialog box, from the **Attribute** list, select one or more items as target attributes to map data to. You can also scroll down to the **IRE Settings** section of the list and select one of the robust import set transformer properties . Click **Save**.
For information about precedence order between robust import set transformer properties defined at the individual item level

and at the IRE payload level, see robust import set transformer properties .

c. Map any lookup rules such as the 'Serial Number Lookup 1' rule.

   Lookup rules are in a deactivated state until you map them. Click the filter icon for the lookup rule to edit or add any lookup filters. In the lookup filter dialog box, specify attribute or plugin conditions that must be met for data to be mapped to various target classes. Then click **Save**.

   After mapping a field of a lookup rule, you can set the Activate/ Deactivate Lookup rule toggle switch for a rule, to on or off.

d. (Optional) Click **View Class Details** to view the current class in CI Class Manager.

e. (Optional) Click the **Transform Data** tab to navigate to the data preparation page where you can review and further transform data that you want to map.

f. Return to the Select CMDB Classes to Map Source Data page.

7. Click **Mark as Complete**.

Add Relationships

Add relationships that exist among the target CMDB classes, for an integration.

Before you begin

- A class that you want to add in the relationship, must be in an activated state.
- A base relationship or a relationship within a conditional relationship, that you want to edit, must be in an activated state.
- In a conditional relationship that you want to edit, at least one relationship condition must be in an activated state. Otherwise, the **Edit Relationship** button is grayed out.

Role required: cmdb_inst_admin

About this task

When creating relationships with nested data, you can't create a relationship between sibling objects from the nested data. Using the sample payload for nested data as an example, you can't create a relationship between interfaces and software.

ITOM Visibility, if available, uses enhanced discovery patterns to identify and add CI relationships to the Suggested Relationships table in the base system. When applicable, use the Suggested Relationships table to select relationships that are in compliance with Common Service Data Model (CSDM) standards.

Procedure

1. Navigate to **All > Configuration > IntegrationHub ETL**, and click the **Name** of an integration.
   The landing page of the IntegrationHub ETL lists all integrations that exist in the system, including integrations that were downloaded from the ServiceNow Store.

2. In the ETL Transform Map Assistant page, in the Map Data to CMDB and Add Relationships section of the guided setup, select **Add Relationships**.

3. To add relationships, select **Add Relationship** or **Add Conditional Relationship** if you want to specify attribute conditions that must be met before adding a relationship. Then, complete the following actions as needed.

Option	Description
Add Relationship	a. Select the **Parent**, **Child**, and **Relationship Type** values. b. Click **Add**.
Add Conditional Relationship	a. In the **choose field** list, select attribute conditions that the data values must meet. b. Select the **Parent**, **Child**, and **Relationship Type** values.

Option	Description
	c. Click **Save**. When processing nested data, a prefix denotes the first level in the nested hierarchy for attribute items.

The **Relationship Type** list menu changes based on the selected parent and child class:

- If there is a dependent relationship, the list is disabled and the relationship type is automatically populated.
- If there is more than one dependent relationship, the list displays both containment and hosting relationship options and the containment relationship type is automatically populated.
- If there is no dependent relationship, the list displays **Suggested relationships** with the first suggested relationship automatically selected, followed by the base system relationship types.
- If there is no suggested relationship, the list displays **No suggested relationships** followed by the base system relationship types.

4. Click **Save** to save the current changes or **Mark as Complete**.

   A time stamp appears in the header when you click **Save**, which remains for the duration of the Integration Hub ETL session for the ETL transform map. When you re-enter the session or switch between ETL maps, the time stamp disappears.

Preview mapping results

Preview the results of the sample data integration.

Before you begin

Role required: cmdb_inst_admin

About this task

Run an integration test and view a summary of the results, for the sample data (by default, up to 100 records). The summary includes total numbers for relationships that were created, mapped classes, partial and incomplete payloads that IRE couldn't process. You can also view detailed messages from Robust Transform Engine (RTE) and from Identification Reconciliation Engine (IRE).

> **Note:** Most IntegrationHub ETL log messages (from RTE and IRE) are informational. However, even if the com.glide.import_set.importlog_level and the glide.importlog.log_to_table system properties are set to not add INFO log messages, IntegrationHub ETL does render INFO log messages. For more details about these properties, see Import sets properties.

After you view the details in the summary page, you can return to any step to make adjustments and then rerun the integration.

Procedure

1. Navigate to **All > Configuration > IntegrationHub ETL**, and click the **Name** of an integration.
   The landing page of the IntegrationHub ETL lists all integrations that exist in the system, including integrations that were downloaded from the ServiceNow Store.

2. In the ETL Transform Map Assistant page, in the Preview Sample Integration Results and Schedule Import section of the guided setup, select **Test and Rollback Integration Results**.

3. On the Test and Rollback Integration Results page, click **Run Integration**.

4. View the summary page and click the various tabs to see the integration run results for the affected CMDB classes. You can click  to open CI forms and view information.

Note: The order of the attribute columns follows the default columns list for the class in the platform. First, the default columns for the class appear from left to right, followed by the rest of the attribute columns organized in alphabetical order. For example, to see the default columns list for the Computers class, navigate to **All > Configuration > Computers**.

5. (Optional) Select any class tab and click **Edit Mapping** to return to the Select CMDB Classes to Map Source Data page where you can review and change mapping settings.

   Note: Clicking **Edit Mapping** rolls back all the changes that were made to the CMDB as a result of this integration run.

6. (Optional) Click the **Relationships** tab and review any relationships that were created.

7. (Optional) Click **Edit Relationships** to return to the Add Relationships page where you can review and change any relationship configurations.

   Note: Clicking **Edit Relationships** rolls back all the changes that were made to the CMDB as a result of this integration run.

8. Click the **Error Log**, **Activity Log**, or the **Warning Log** tabs to see the respective details logged by IRE and RTE during the integration.

   IRE log records are grouped by categories and further organized by the respective class. For IRE log messages, the Message column contains only the messages themselves which were extracted from the raw log message. The Log Message column contains the complete log message, which includes class and category in addition to the message itself. RTE logs appear under the Other category.

   Use the **Verbose** toggle switch to change the viewing mode for the Message and the Log Message columns:

   - Verbose on: Shows fully expanded text of log messages.
   - Verbose off: Shows a condensed version of the log messages. The fully expanded text of the log messages appears when you point to a message.

9. Click the **Incomplete Payloads** and **Partial Payloads** tabs for details about IRE payloads for the integration run.
10. Select **Mark as Complete**.
    The Rollback options dialog box appears and you can choose either of the following options.
    - **Retain Data**: All the changes to the CMDB resulting from this integration, are retained.
    - **Perform Rollback**: All the changes to the CMDB resulting from this integration, are rolled back and the CMDB is restored to its state before running the integration.

Provide integration schedule

Configure a schedule for importing data to CMDB using this ETL Transform Map.

Before you begin

Role required: cmdb_inst_admin

Procedure

1. Navigate to **All > Configuration > IntegrationHub ETL**, and click the **Name** of an integration.
   The landing page of the IntegrationHub ETL lists all integrations that exist in the system, including integrations that were downloaded from the ServiceNow Store.
2. In the ETL Transform Map Assistant page, in the Preview Sample Integration Results and Schedule Import section of the guided setup, select **Set Import Schedule**.
3. On the Provide Schedule page, click **Set Schedules**.
4. In the Scheduled Data Imports list view (which opens in a new tab), click **New**.
5. Fill out the Scheduled Data Import form and then click **Submit**.
   See Schedule a data import for details about the form fields.
6. Click **Mark as Complete**.

Transform types in IntegrationHub ETL

Use various transforms in IntegrationHub ETL to convert and prepare source data for mapping to the CMDB.

Transforms from the Integration Commons for CMDB store app, are also available in IntegrationHub ETL.

Concatenation

Combines the values from input fields into a single string, joining them on the optional joining_string field.

Details	
Table	sys_rte_eb_concat_operation
Input fields	source_sys_rte_eb_fields
Output field	target_sys_rte_eb_field
Additional Fields	joining_string (optional)

Example	
Input	“input_1”, “input_2”, “input_3”
Additional Fields	joining_string = ", "
Result	“input_1, input_2, input_3”

Convert to Boolean

Converts the incoming value to a boolean. ‘`true`’ and ‘`1`’ values convert to 'true' (case insensitive), and any other values convert to 'false'.

Details	
Table	sys_rte_eb_to_boolean_operation
Input fields	source_sys_rte_eb_field
Output field	target_sys_rte_eb_field

Examples:

- All of the following inputs return 'true':
  - true
  - 1
- All of the following inputs return 'false':
  - “input_1”
  - “”
  - 0
  - 11

Convert to Date

Attempts to convert the incoming value to a GlideDateTime value by applying the date_format to the incoming value. Attempts to directly convert using GlideDateTime if the date_format is incorrect.

Details	
Table	sys_rte_eb_to_date_operation
Input fields	source_sys_rte_eb_field
Output field	target_sys_rte_eb_field Returns an empty value if unable to parse at all.

Details	
Additional Fields	date_format (Java simple date format)

Example	
Input	"2018/09/20 11:21:00 a.m. EST"
Additional Fields	date_format = "yyyy/MM/dd hh:mm:ss a z"
Result	"2018-09-20 16:21:00"

Example	
Input	"2018/09/20 01:21:00 PM EST"
Additional Fields	date_format = "yyyy/MM/dd hh:mm:ss a z"
Result	"2018-09-20 18:21:00"

Example	
Input	"09/20/18"
Additional Fields	date_format = "yyyy/MM/dd hh:mm:ss a z"
Result	"0018-09-20 00:00:00"

Convert to Numeric

Converts the incoming value to a number.

Details	
Table	sys_rte_eb_to_numeric_operation
Input fields	source_sys_rte_eb_field
Output field	target_sys_rte_eb_field If the incoming value is non-numeric, then the output is empty.

Example	
Input	1.23
Result	1.23

Example	
Input	1.00
Result	1

Example	
Input	input_1
Result	null

Example	
Input	two
Result	null

Copy

Copies the source field's value to all of the target fields.

Details	
Table	sys_rte_eb_copy_operation
Input fields	source_sys_rte_eb_field
Output field	target_sys_rte_eb_fields
Additional Fields	overwrite_existing_value (optional, boolean): If **true**, then the values of target fields are replaced. Otherwise, any non-empty value is not overwritten.

Extract Leading Numeric

Sets the target field to be the first numeric value found in the source field.

Details	
Table	sys_rte_eb_extract_numeric_operation
Input fields	source_sys_rte_eb_field
Output field	target_sys_rte_eb_field
Additional Fields	• decimal_places (optional, number): Forces the output to have a specified number of decimal places. • remainder_target_field (optional, reference to a field): Set to the trimmed remainder of the source field, after removing the first numeric value.

Example	
Input	“100 mb”
Result	"100”

Example	
Input	“100.123 mb”
Result	“100.123"

Example	
Input	“100.123 mb”
Additional Fields	decimal_places = 2
Result	“100.12”

Example	
Input	“100 mb”
Additional Fields	decimal_places = 2
Result	“100.00”

Example	
Input	“100 mb”
Additional Fields	remainder_target_field = <field>

Example	
Result	“100” and <field> = “mb”

Glide Lookup

Performs a lookup in the database on the target_table.

Details	
Table	sys_rte_eb_glide_lookup_operation
Input fields	source_sys_rte_eb_fields
Output field	target_sys_rte_eb_fields
Additional Fields	• target_table • glide_matching_fields (string): Comma-separated list of column names in the target table. For each input field in source_sys_rte_eb_fields, there must be an equal number of values in glide_matching_fields • glide_target_fields (string): Comma-separated list of column names in the target table. For each target field in target_sys_rte_eb_fields, there must be an equal number of values in glide_target_fields.

Example	
Input	• Input Field 1: 100 South Charles Street, Baltimore • Input Field 2: MD
Additional Fields	• Target Table: Location (cmn_location)

Example	
	• Glide Matching Fields: street,state • Glide Target Fields: sys_id
Result	Output Field 1: 25ab9c4d0a0a0bb300f7dabdc0ca7c1c

Min/Max

Sets the target field to either the maximum or minimum of the values from all input fields.

Details	
Table	sys_rte_eb_min_max_operation
Input fields	source_sys_rte_eb_fields
Output field	target_sys_rte_eb_field
Additional Fields	• data_type (choice list <STRING,NUMERIC,DATE>) • min_max (choice list <MIN,MAX>)

Example	
Input	“2”, “-1”, “0”
Additional Fields	• data_type = NUMERIC • min_max = MAX
Result	“2”

Example	
Input	“a”, “b”
Additional Fields	• data_type = STRING • min_max = MAX
Result	“b”

Example	
Input	“2”, “-1”, “0”
Additional Fields	• data_type = NUMERIC • min_max = MIN
Result	“-1”

Example	
Input	“a”, “b”
Additional Fields	• data_type = STRING • min_max = MIN
Result	“a”

Multiple Input Script

Runs a script with multiple inputs, setting the target_field == output for that script.

Each source field is available inside of the 'batch' variable as JavScript fields. The name of the JavaScript field is the field attribute of the entity field (looking at sys_rte_eb_field.field, not sys_rte_eb_field.name).

Details	
Table	sys_rte_eb_multi_in_script_operation
Input fields	source_sys_rte_eb_fields
Output field	target_sys_rte_eb_field
Additional Fields	• script (script) • use_unique_input_sets (boolean): When **true**, only unique input values are included in the data batch for IRE processing. Otherwise, all input object's field values are included.

Example for using use_unique_input_sets, with a script function that takes record_type and operating_system as input and returns record_with_os:

Input data

Record	record_type	operating_system	record_with_os
1	computer	Windows XP	
2	computer	Linux	
3	computer	Windows XP	

If use_unique_inputs_sets is set to **true**, then the script processes only two values (computer + Windows XP and computer + Linux). If use_unique_inputs_sets is set to **false**, then each of the three values is individually processed (computer + Windows XP, computer + Linux, and computer + Windows XP).
Sample script:

```
(function(batch, output) {
    for (var i = 0; i < batch.length; i++) {
            // batch[i] is the unique set of
```

```
inputs/individual record
                        // batch[i].<field> gives access t
o the field value
                        var in0 = gs.nil(batch[i].record_
type) ? '' : batch[i].record_type;
                        var in1 = gs.nil(batch[i].operati
ng_system) ? '' : batch[i].operating_system;
                        // output[i] is the output for th
e specific combination of inputs/individual record
                        output[i] = in0 + "_" + in1;
                    }
                }
            })(batch, output);
```

Sample script:

```
            /* Example Script
             // In this example the script input fields a
re 'input_field_1', 'input_field_2' - replace these with th
e fields used as script inputs // There is a static field '
input' that has all the input field values concatenated wi
th a '|' (function(batch, output) {
                for (var i = 0; i < batch.length; i++) {

                    //step1: access the input variables
                    var a = batch[i].input_field_1; //Val
ue of the first source field.
                    var b = batch[i].input_field_2; //Val
ue of the second source field.

                    //step2: Your script/code goes here.
                    var c = a + b;

                    //step3: set the output for each ele
ments
                    output[i] = b;
                }

            })(batch, output);
           */
```

Rexeg Replace

Replaces each substring of the incoming string that matches the specified match_regex, with the specified replacement_regex string value.

Details	
Table	sys_rte_eb_regex_replace_operation
Input fields	source_sys_rte_eb_field
Output field	target_sys_rte_eb_field
Additional Fields	• match_regex (string, regular expression) • replacement_regex (string)

Example	
Input	“String&With(Special)$Characters”
Additional Fields	• match_regex = “[^0-9a-zA-Z]+” • replacement_regex = “ ”
Result	“String With Special Characters”

Replace

Replaces each substring in the incoming string that matches the specified match_string, with the replacement_string string value.

Details	
Table	sys_rte_eb_replace_operation
Input fields	source_sys_rte_eb_field
Output field	target_sys_rte_eb_field
Additional Fields	• match_string (string) • replacement_string (string)

Example	
Input	"Original String"
Additional Fields	• match_string = "Original" • replacement_string = "Replacement"
Result	"Replacement String"

Round Numeric

Rounds the number value to the nearest whole number. Non-numbers are truncated.

Details	
Table	sys_rte_eb_round_numeric_operation
Input fields	source_sys_rte_eb_field
Output field	target_sys_rte_eb_field

Example	
Input	“1.5”
Result	"2"

Example	
Input	“1.4”
Result	"1"

Example	
Input	“i’m a string”
Result	""

Script

Runs a script with input, setting the target_field == output for that script.

This transform has been superseded by the Multi Input Script transform and is included for backwards compatibility with existing configurations.

Details	
Table	sys_rte_eb_script_operation
Input fields	source_sys_rte_eb_field
Output field	target_sys_rte_eb_field
Additional Fields	• script (script) • use_unique_input_sets (boolean): When **true**, only unique input values are included in the data batch for

Details	
	IRE processing. Otherwise, all input object's field values are included. For an example and for more details, see the Multiple Input Script transform.

The source field is included in the 'batch' variable as the JavaScript field 'input'.

```
          (function(batch, output) {
                for (var i = 0; i < batch.length; i++) {
                        // batch[i] is the unique set of
inputs/individual record
                        // batch[i].input gives access t
o the field value
                        var in0 = gs.nil(batch[i].input)
? '' : batch[i].input;
                        // output[i] is the output for th
e specific combination of inputs/individual record
                        output[i] = in0 + " modified by sc
ript";
                    }
                }
            })(batch, output);
```

Example:

```
/* Example Script
 (function(batch, output) {
     for (var i = 0; i < batch.length; i++) {
         //step1: access the input variables
         var a = batch[i].input; //Value of the source fie
ld.

         //step2: Your script/code goes here.
         var b = a + 1;
         //step3: set the output for each elements
         output[i] = b;
     }
 })(batch, output);
*/
```

Set

Sets the target field's value to the string specified in set_value.

Details	
Table	sys_rte_eb_set_operation
Input fields	source_sys_rte_eb_field
Output field	target_sys_rte_eb_field
Additional Fields	• set_value (string) • overwrite_existing_value (optional, boolean): When **true**, the current value of the target field is overwritten. Otherwise, a non-empty value isn't replaced.

Split

Splits the source field's value on the splitting_string and assigns each resulting item from the split to the target_sys_rte_eb_fields, in order.

Details	
Table	sys_rte_eb_split_operation
Input fields	source_sys_rte_eb_field
Output field	target_sys_rte_eb_fields
Additional Fields	splitting_string (string)

Example	
Input	"value1||value2||value3", with target_sys_rte_eb_fields {target1,target2,target3}
Additional Fields	splitting_string = "||"

Example	
Result	target1 : value1, target2 : value2, target3 : value3

Example	
Input	“value1||value2||value3”, with target_sys_rte_eb_fields {target1}
Additional Fields	splitting_string = ”||”
Result	target1 : value1

Example	
Input	“value1”, with target_sys_rte_eb_fields {target1,target2,target3}
Additional Fields	splitting_string = ”||”
Result	target1 : value1, target2 : <null>, target3 : <null>

Trim

Trims leading and trailing whitespace from the source_sys_rte_eb_field value and assigns the result to the target_sys_rte_eb_field. This transform is equivalent to a Java String.trim().

Details	
Table	sys_rte_eb_trim_operation
Input fields	source_sys_rte_eb_field
Output field	target_sys_rte_eb_field

Example	
Input	“ value 1 ”
Result	“value 1”

Uppercase

Uppercases the source_sys_rte_eb_field value and assigns the result to target_sys_rte_eb_field.

Details	
Table	sys_rte_eb_upper_case_operation
Input fields	source_sys_rte_eb_field
Output field	target_sys_rte_eb_field

Example	
Input	“value1”
Result	“VALUE1”

Uppercase Trim

Combines both the Uppercase and the Trim transforms.

Details	
Table	sys_rte_eb_upper_case_trim_operation
Input fields	source_sys_rte_eb_field
Output field	target_sys_rte_eb_field

Example	
Input	“ value1 ”
Result	“VALUE1”

Add before and after scripts

Add custom before and after scripts for a data source of a CMDB integration application. Those scripts provide access to the input and output payloads of IRE. When a CMDB integration invokes Identification and Reconciliation Engine (IRE), those scripts run before and after IRE processes the integration payload.

Before you begin

Role required: cmdb_inst_admin

Procedure

1. Navigate to **All > Configuration > IntegrationHub ETL**.

   The landing page of the IntegrationHub ETL lists all integrations that exist in the system, including integrations that were downloaded from the ServiceNow Store.

2. Click the **CMDB Application** link for the integration that you want to add scripts for.

3. On the CMDB Integration Studio Application form, in the CMDB Integration Studio Application Data Sources related list, open the data source record for the CMDB integration.

4. On the CMDB Integration Studio Application Data Source form, check **Execute Before Script** or **Execute After Script**. Review the **Before Script** and **After Script** comments and enter your custom scripts at the bottom of the script field.
   The comments in the script fields provide details for the before and after scripts. Use those guidelines, explanations such as what operations are supported, and examples when creating your custom script.

5. Click **Update**.

Related concepts

- Identification and Reconciliation engine (IRE)

Duplicate an ETL transform map

Use an existing ETL transform map if you need to create a map that is mostly similar to that existing map. Select an existing map, specify a new data source for the new duplicate map, and then change or retain other settings such as data transforms.

Before you begin

The data source for the new duplicated ETL transform map must satisfy these requirements:

- The schema of the data source must be identical to the schema of the original ETL transform map. For example, both data sources must have an identical number of columns and identical column labels.
- The data source must preexist.
- The data source must not be used in any other ETL transform map.

Role required: cmdb_inst_admin

Procedure

1. Navigate to **All > Configuration > IntegrationHub ETL**.
   The landing page of the IntegrationHub ETL lists all integrations that exist in the system, including integrations that were downloaded from the ServiceNow Store.
2. Click **Duplicate** and then fill out the Duplicate ETL Transform Map form.

Field	Description
Duplicate from	The ETL transform map to duplicate from.

Field	Description
CMDB application	The CMDB application associated with the ETL transform map. You can select **Add new**, which adds the **CMDB Application** and the **Discovery Source** fields for the new CMDB application.
Discovery Source	Discovery source associated with a new CMDB Application. Appears if you set **CMDB Application** to **Add new**.
CMDB Application Name	Name of a new CMDB Application. Appears if you set **CMDB Application** to **Add new**.
Name	Name of the ETL transform map.
Data Source	The source feed, unique for this ETL transform map, where the raw source data is imported from.

3. Click **Create Duplicate**.

Result

Data for the new duplicated ETL transform map is imported, and the **Import Source Data and Provide Basic Details** task, is set as complete.

What to do next

Continue with the next steps on the guided setup steps to review the imported data and complete the integration using the duplicated ETL transform map.